

移动与嵌入式操作系统深度研究报告：架构原理、生态挑战与未来演进

1. 引言：非传统操作系统的计算范式

在现代计算技术的宏伟版图中，通用桌面操作系统（如 Windows、macOS 和桌面 Linux）长期占据着传统的中心地位，然而，随着泛在计算（Ubiquitous Computing）时代的到来，计算能力的边界已经远远延伸至传统的个人电脑之外。本报告基于“非传统操作系统（Non-Conventional OS）”这一核心议题，结合项目要求与演示文稿（PPT）的框架，对**移动操作系统（Mobile OS）与嵌入式操作系统（Embedded OS）**进行详尽的深度剖析。

这两类操作系统虽然在定义上有所区分，但共同构成了连接物理世界与数字世界的关键基础设施。移动操作系统，以 Android 和 iOS 为代表，重新定义了人机交互的方式，将高性能计算封装于掌中；而嵌入式操作系统，如 VxWorks、QNX、FreeRTOS 及嵌入式 Linux，则在工业自动化、医疗设备、汽车电子及物联网（IoT）中扮演着隐形但至关重要的控制角色。本报告将严格遵循学术与行业分析的高标准，通过对架构设计、资源管理机制、安全性演进及未来趋势的深入探讨，揭示这两大领域的内在逻辑与发展脉络。

2. 移动操作系统：高性能与移动性的架构平衡

移动操作系统（Mobile OS）是专为智能手机、平板电脑、可穿戴设备等移动终端设计的系统软件平台¹。与传统桌面系统相比，移动 OS 必须在极其严苛的功耗限制（Thermal Design Power, TDP）和电池容量约束下，提供流畅的触摸交互体验、持续的网络连接以及复杂的应用生态支持。

2.1 市场格局与核心架构解析

当前移动操作系统市场呈现出明显的双寡头垄断格局，即 Google 主导的 Android 生态与 Apple 主导的 iOS 生态。这两者在设计哲学上的根本差异——开放与封闭、宏内核与混合内核——决定了它们在功能实现与系统演进上的不同路径。

2.1.1 Android：基于 Linux 的宏内核改良与生态开放

Android 并非一个单纯的操作系统，而是一个基于 Linux 内核的完整软件栈。其核心架构设计体现了对开源协作与硬件抽象的高度重视。

Linux 内核层的定制化演进

Android 的底层依赖于 Linux 内核，利用其成熟的进程管理、内存管理和驱动模型。然而，为了适应移动设备的特殊需求，Google 对标准 Linux 内核进行了深度修改。其中最显著的差异在于进程间通信（IPC）机制。标准 Linux 使用管道（Pipes）或消息队列，而 Android 引入了 Binder 驱动。Binder 提供了一种高效的远程过程调用（RPC）机制，使得不同进程能够以极低的开销共享对象和数据，这对于 Android 这种组件化极强的系统至关重要²。此外，Android 还引入了 Ashmem（匿名共享内存）机制，优化了低内存环境下的资源分配效率。

硬件抽象层（HAL）的战略意义

在内核之上, Android 设计了硬件抽象层(HAL)。这一层的存在主要是为了解决 Linux 的 GPL 开源协议与硬件厂商(如高通、联发科)保护专有驱动代码(IP)之间的矛盾。HAL 定义了一套标准的接口(.h 文件), 硬件厂商只需提供编译好的二进制库(.so 文件)来实现这些接口, 而无需将驱动源码公开合并入内核。这种设计极大地促进了 Android 在不同硬件平台上的广泛适配²。

Android Runtime (ART) 与 Zygote 启动机制

Android 的应用运行环境经历了从 Dalvik 到 ART(Android Runtime)的变革。ART 引入了预编译(AOT)技术, 在应用安装时即将其字节码编译为本地机器码, 从而在运行时大幅提升执行效率并降低 CPU 功耗⁵。在启动流程中, Zygote 进程扮演着核心角色。系统启动时, Zygote 进程会预加载所有核心库和类资源。每当用户启动一个新应用, Zygote 并不是从头加载, 而是直接“分叉”(fork)自身来创建一个新进程。这种写时复制(Copy-on-Write)技术使得所有应用能够共享核心内存页, 极大地缩短了应用启动时间(App Startup Time)并节省了系统内存³。

2.1.2 iOS: XNU 混合内核与垂直整合的极致

与 Android 的开源策略截然不同, iOS 建立在 Apple 独有的硬件生态之上, 其系统架构强调软硬件的极致协同。

XNU 内核的混合架构

iOS 的核心是 XNU(X is Not Unix)内核, 这是一种混合内核架构, 融合了 Mach 微内核的消息传递机制与 BSD 的宏内核服务(如文件系统、网络协议栈)。这种设计试图在微内核的模块化与宏内核的性能之间寻找平衡⁶。XNU 内核之上是 Darwin 层, 提供了系统的基础服务。

Cocoa Touch 与应用沙盒

iOS 的应用层框架 Cocoa Touch 强制执行严格的 MVC(Model-View-Controller)设计模式, 并对用户界面(UI)渲染给予最高优先级。在安全性方面, iOS 采用了强制性的沙盒机制(Sandboxing)。每个应用都在独立的容器中运行, 无法直接访问其他应用的数据或系统核心文件。这种设计虽然限制了应用间的直接交互(需通过 URL Scheme 或 App Extensions 进行), 但极大地遏制了恶意软件的横向传播能力⁷。

2.2 移动操作系统的关键特性与功能优先级

移动 OS 的设计必须优先考虑以下几个关键维度, 以满足现代用户的高标准需求¹:

- 人机交互优化: 移动 OS 的核心竞争力在于对触摸屏界面的优化。无论是 iOS 的惯性滚动还是 Android 的手势导航, 系统必须以 60Hz 甚至 120Hz 的刷新率响应用户操作, 任何掉帧或延迟都会被视为严重的体验缺陷。
- 电源效率与云服务集成: 鉴于电池容量的物理限制, 移动 OS 内置了复杂的电源管理策略(如 Android 的 Doze 模式)。同时, 深度集成的云服务(iCloud, Google Drive)使得数据同步和备份成为系统级功能, 而非附加应用。
- 无缝连接能力: 移动设备必须支持蓝牙、Wi-Fi、NFC 和蜂窝网络(4G/5G)的即时切换与并发工作, 这要求 OS 底层网络栈具有极高的鲁棒性和自适应能力。

2.3 移动操作系统面临的挑战与弱点

尽管移动 OS 已经发展得相当成熟, 但仍面临着一系列根深蒂固的技术与市场挑战, 这些挑战在 PPT 资料中被重点提及, 并在当前的技术演进中愈发凸显。

2.3.1 碎片化(Fragmentation):生态系统的阿喀琉斯之踵

碎片化是 Android 生态系统面临的巨大挑战,也是其开放特性带来的必然副作用。

- 现象描述:碎片化指的是市场上同时运行着大量不同版本的 Android 系统,且这些系统往往经过了设备制造商(OEM)和电信运营商的深度定制(如 Samsung OneUI, Xiaomi HyperOS)。这导致了用户体验的不一致,开发者必须为成百上千种屏幕尺寸、分辨率和 API 级别进行适配¹。
- 数据佐证:截至 2025 年 4 月的数据显示,尽管 Android 15 已经发布,但市场上仍有约 27.4% 的设备运行 Android 14, 16.8% 运行 Android 13,甚至还有大量设备停留在 Android 11 及更早版本⁹。这种版本分布的长尾效应极大地增加了应用开发和维护的成本。
- 解决机制的演进:为了缓解这一问题,Google 先后推出了 **Project Treble** 和 **Project Mainline**。
 - **Project Treble** 通过引入供应商接口(Vendor Interface),将 Android OS 框架与供应商实现(Vendor Implementation)解耦。理论上,这意味着 OS 框架可以独立更新,而无需芯片厂商重新编写驱动¹⁰。
 - **Project Mainline** 则更进一步,将操作系统的核心组件(如媒体编解码器、DNS 解析器、安全模块)模块化。这些模块打包为 APEX 或 APK 文件,可以直接通过 Google Play 商店进行更新,就像更新普通 App 一样。这使得 Google 能够绕过 OEM,直接向数十亿用户推送关键的安全补丁和功能更新¹²。

2.3.2 安全漏洞与隐私困境

移动设备存储了用户的地理位置、金融信息、健康数据等极度敏感的个人数据,使其成为网络攻击的首选目标。

- 安全威胁:移动 OS 面临着恶意软件(Malware)、钓鱼攻击(Phishing)和数据泄露的持续威胁。尤其是 Android 平台,由于允许从第三方来源安装应用(Sideload),用户更容易接触到未经审核的恶意代码¹。
- 隐私博弈:应用程序往往过度索取权限(如手电筒应用请求通讯录权限)。虽然 Android 和 iOS 都引入了运行时权限请求(Runtime Permissions)和隐私仪表盘(Privacy Dashboard),但在提供便捷服务(如基于位置的推荐)与保护用户隐私之间,始终存在着难以调和的矛盾¹⁴。
- 架构级防御:现代移动 OS 引入了基于硬件的安全机制。
 - **Apple Secure Enclave (SEP)**:这是一个独立的协处理器,拥有自己的微内核(L4)和加密内存,专门用于处理指纹、面容 ID 数据和加密密钥。即使 iOS 内核被攻破,攻击者也无法从 SEP 中提取密钥¹⁶。
 - **ARM TrustZone**:Android 设备普遍采用 TrustZone 技术,将 CPU 划分为“安全世界”和“正常世界”。敏感操作在安全世界的可信执行环境(TEE)中运行,与普通 Android 系统隔离¹⁶。

2.3.3 资源限制与性能优化的永恒战争

尽管移动处理器的性能已逼近桌面级 CPU,但散热和电池技术的停滞使得资源优化依然是核心挑战。

- 电池寿命优化：高性能应用和后台进程 (Background Processes) 是电池杀手。Android 的 **Doze 模式** 和 **App Standby** 机制通过在设备闲置时强制暂停网络访问和延迟后台任务来节省电量¹⁹。iOS 则通过严格限制后台活动，仅允许特定的 API (如 VoIP、音频播放、位置更新) 在后台运行，从而保证续航。
- 存储限制与内存管理：移动设备的存储空间有限且不易扩展。随着应用体积的膨胀 (App Bloat) 和高清媒体内容的普及，存储管理变得日益困难。此外，移动 OS 不像桌面系统那样依赖交换空间 (Swap Space)，当内存不足时，系统会根据优先级 (Low Memory Killer) 直接杀掉后台进程，这对应用的状态保存和恢复机制提出了极高要求²¹。

3. 嵌入式操作系统：确定性与效率的极致追求

与移动操作系统的通用性不同，嵌入式操作系统 (Embedded OS) 是为特定硬件和特定任务而生的。它们广泛应用于工业控制、汽车电子、航空航天和医疗设备中，其核心价值不在于多功能，而在于确定性 (**Determinism**)、可靠性 (**Reliability**) 和资源效率 (**Resource Efficiency**)¹。

3.1 嵌入式系统的分类与架构特征

嵌入式 OS 主要分为两大类：实时操作系统 (**RTOS**) 和嵌入式 **Linux** (及其他通用 OS 的嵌入式版本)。

3.1.1 实时操作系统 (RTOS)：时间的绝对掌控者

RTOS 的核心特征是能够保证任务在规定的时间内完成。对于心脏起搏器或汽车安全气囊控制器而言，延迟意味着生命危险。

- 调度算法与确定性：RTOS 通常采用基于优先级的抢占式调度 (**Priority-based Preemptive Scheduling**)。这意味着一旦高优先级任务 (如传感器中断) 准备就绪，调度器会立即暂停当前运行的低优先级任务，切换上下文执行高优先级任务。这种切换的延迟 (Latency) 通常在微秒 (μ s) 级别²²。相比之下，通用 OS (如标准 Linux) 采用完全公平调度 (CFS)，旨在最大化吞吐量而非响应速度，其延迟可能达到毫秒级且不可预测²⁴。
- 内存管理策略：为了避免运行时内存分配失败或碎片化导致的性能抖动，许多安全关键型 RTOS (如 FreeRTOS, SafeRTOS) 倾向于使用静态内存分配。所有的内存需求在编译时即确定，系统运行时不进行 malloc/free 操作。此外，微控制器 (MCU) 通常只配备**内存保护单元 (MPU)** 而非内存管理单元 (MMU)，MPU 仅能提供基本的区域保护，而不支持虚拟内存和分页技术，这进一步简化了系统但也限制了其复杂性²⁵。
- 典型代表：
 - **FreeRTOS**：市场占有率极高，以其极小的内核 (<10KB) 和开源特性成为微控制器领域的首选。它提供基本的任务调度、队列和信号量机制²⁷。
 - **VxWorks**：广泛应用于航空航天 (如火星探测器) 和工业机器人。它提供硬实时保证和极高的可靠性认证 (如 DO-178C)²⁹。
 - **Zephyr**：由 Linux 基金会托管的新一代开源 RTOS，支持模块化构建和广泛的硬件架构。它引入了类似 Linux 的设备树 (Device Tree) 配置机制，并在物联网和边缘计算领域迅速崛起³⁰。

3.1.2 嵌入式 Linux: 灵活性与功能的平衡

对于需要复杂网络协议栈、图形用户界面(GUI)或高级文件系统的嵌入式设备(如智能网关、车载娱乐系统), RTOS 往往力不从心, 此时嵌入式 Linux 成为主流选择。

- **Yocto Project** 与构建系统: 嵌入式 Linux 并非单一发行版, 而是通过构建系统(如 Yocto Project, Buildroot)生成的定制化镜像。开发者可以裁剪内核, 只保留必要的驱动和工具, 从而在资源受限的设备上运行完整的 Linux 系统³²。
- **实时性补丁(PREEMPT_RT)**: 虽然标准 Linux 不是实时的, 但通过打上 PREEMPT_RT 补丁, 可以将内核中的大部分自旋锁(Spinlocks)变为互斥锁, 使内核代码段变为可抢占, 从而显著降低延迟, 使其接近硬实时性能³⁴。

3.2 嵌入式操作系统的优劣势剖析

根据 PPT 内容及研究资料, 嵌入式 OS 的优缺点极其鲜明。

3.2.1 核心优势(Advantages)

1. **资源效率(Resource Efficiency)**: 嵌入式 OS 经过高度优化, 能够在仅有几 KB 内存和几十 MHz 主频的 MCU 上运行。例如, FreeRTOS 的内核代码量极小, 几乎不占用额外的 Flash 空间¹。
2. **实时操作(Real-Time Operation)**: RTOS 提供的确定性响应是工业自动化和医疗设备不可或缺。例如, Da Vinci 手术机器人必须在毫秒级时间内将医生的手部动作转化为机械臂的运动, 且不能有任何抖动(Jitter)³⁶。
3. **定制化与专用解决方案(Customization)**: 开发者可以根据硬件特性完全掌控系统的每一个字节, 去除所有冗余功能, 实现极致的精简¹。
4. **可靠性与稳定性(Reliability)**: 嵌入式系统通常需要 7x24 小时连续运行数年而不重启。其简化的架构减少了软件崩溃的概率, 看门狗(Watchdog)机制则确保系统在异常时能自动复位¹。
5. **低功耗(Low Power Consumption)**: 通过 Tickless Idle(无滴答空闲)技术, RTOS 可以在系统空闲时停止周期性的时钟中断, 让 CPU 进入深度睡眠模式, 直到下一个任务预定时间或外部中断到来。这对于电池供电的 IoT 传感器至关重要³⁸。

3.2.2 劣势与挑战(Disadvantages)

1. **硬件支持受限(Limited Hardware Support)**: 与通用 OS 拥有海量驱动库不同, 嵌入式 OS 往往需要开发者自行编写底层驱动, 硬件兼容性较差¹。
2. **开发过程复杂(Complex Development Process)**: 缺乏高级抽象, 开发者需要直接操作寄存器和内存, 调试难度大, 且需要深厚的计算机体系结构知识。开发周期长, 成本高¹。
3. **可扩展性挑战(Scalability Challenges)**: 一旦硬件选定且软件架构固化, 后期想要增加新功能(如从单核升级到多核, 或增加复杂的网络协议)往往需要重写大量代码¹。
4. **有限的用户界面(Limited User Interface)**: 大多数嵌入式 OS 不支持复杂的图形库(如 OpenGL), 仅提供简单的文本或 2D 图形接口, 限制了交互体验¹。
5. **安全性隐忧(Security Concerns)**: 许多老旧的嵌入式系统在设计之初未考虑联网安全, 缺

乏固件更新机制(OTA)。一旦接入物联网,极易成为僵尸网络的一部分。欧盟《网络弹性法案》(Cyber Resilience Act)的实施正迫使这一领域发生变革,要求所有联网设备必须具备“设计安全”和全生命周期的漏洞管理⁴⁰。

4. 融合与演进:行业垂直领域的操作系统变革

随着技术的进步,移动 OS 与嵌入式 OS 的界限正在模糊,两者在汽车、医疗和工业领域的融合应用成为新的趋势。

4.1 汽车电子:Android Automotive 与 QNX 的共生

现代汽车座舱是多操作系统共存的典型场景。

- **Android Automotive OS (AAOS)**: 不同于手机投射的 Android Auto, AAOS 是直接运行在车机硬件上的完整操作系统。它接管了信息娱乐系统(IVI),提供地图、语音助手和应用生态。然而,由于 Android 并非实时系统,它不能用于控制刹车或引擎⁴²。
- **QNX 的安全底座**: 为了保证车辆安全,底层往往运行着 QNX 这样的微内核 RTOS。通过虚拟化技术(Hypervisor),QNX 可以作为宿主系统,同时运行一个负责仪表盘和 ADAS(高级驾驶辅助系统)的实时虚拟机,和一个负责娱乐的 Android 虚拟机。这种架构确保了即使 Android 系统崩溃,通过 QNX 控制的仪表盘和刹车系统依然能正常工作,实现了功能安全(ISO 26262)与丰富生态的完美结合⁴²。

4.2 医疗机器人:硬实时的生命线

在手术机器人领域,操作系统的选择直接关系到患者的生命安全。

- **低延迟与高可靠性**: 远程手术或辅助手术系统要求“端到端”延迟控制在极低水平(通常低于 100ms 的视觉反馈延迟,微秒级的控制回路延迟)。VxWorks 和 QNX 因其严格的 IEC 62304 医疗软件认证和确定性调度能力,成为 Da Vinci 等手术机器人的首选 OS⁴⁵。
- **触觉反馈的实时性**: 系统必须实时处理力传感器数据并反馈给医生手柄,任何抖动(Jitter)都可能导致医生操作手感失真,造成手术失误³⁶。

4.3 工业自动化与 Matter 协议

工业 4.0 推动了嵌入式系统向互联化发展。

- **Matter 协议的统一**: 物联网长期面临 Zigbee、蓝牙、Wi-Fi 等协议的碎片化问题。Matter 协议的出现提供了一个基于 IP 的统一应用层标准,使得不同品牌的嵌入式设备能够互联互通。Zephyr 和 FreeRTOS 都在积极集成 Matter 协议栈,使得嵌入式 OS 成为智能家居互联互通的基石⁴⁷。
- **TinyML 的兴起**: 随着 AI 向边缘端下沉, TinyML 技术允许在资源极其受限的微控制器上运行机器学习模型。嵌入式 OS 如 Zephyr 通过集成 TensorFlow Lite Micro,使得工业传感器能够本地进行异常检测(如电机震动分析),大大减少了云端数据传输需求⁴⁹。

5. 结论

通过对移动与嵌入式操作系统的深入剖析，我们可以看到，尽管两者在设计初衷上分别指向了“以人为中心的体验”和“以机器为中心的控制”，但它们正处于一个相互借鉴、加速融合的历史进程中。

移动操作系统正在通过模块化 (Project Mainline) 和硬件隔离 (Secure Enclave) 解决其碎片化和安全短板，试图在保持灵活性的同时提升可控性。嵌入式操作系统则在坚守实时性与可靠性底线的同时，通过引入 POSIX 标准、虚拟化技术和高级网络协议 (Matter)，极力提升其连接能力和开发效率。

未来，随着 RISC-V 架构的普及、AI 算力的边缘化以及网络安全法规的收紧，这两类操作系统将继续演进。对于开发者和架构师而言，理解它们各自的架构权衡 (Trade-offs) ——是在 Android 上追求极致的生态丰富度，还是在 VxWorks 上追求绝对的时间确定性——将是构建下一代智能系统的关键所在。

6. 数据对比表

为了更直观地展示各类操作系统的特性差异，以下表格总结了核心参数对比：

表 1: 移动操作系统对比 (Android vs. iOS)

特性	Android	iOS
内核架构	修改版 Linux (宏内核)	XNU (Mach 微内核 + BSD 混合架构)
开源模式	开源 (AOSP), 厂商定制多	闭源 (Proprietary), Apple 独占
硬件抽象	HAL 层, 适配广泛硬件	垂直整合, 仅适配 Apple Silicon
更新机制	碎片化严重, 依赖 Mainline 缓解	集中推送, 更新率高
安全机制	权限模型, 沙盒, TrustZone	围墙花园 (App Store), Secure Enclave
启动流程	Boot ROM -> Bootloader -> Kernel -> Init -> Zygote	Boot ROM -> LLB -> iBoot -> Kernel
IPC 机制	Binder (基于内核驱动)	Mach Messages (微内核原

		语)
--	--	----

表 2: 嵌入式操作系统对比 (RTOS vs. 嵌入式 Linux)

特性	实时操作系统 (RTOS)	嵌入式 Linux
调度策略	基于优先级的抢占式 (确定性)	完全公平调度 CFS (吞吐量优先)
响应延迟	微秒级 (硬实时)	毫秒级 (软实时/非实时)
内存管理	静态分配, MPU (扁平内存)	虚拟内存, MMU (分页/交换)
资源占用	极低 (KB 到 MB 级)	较高 (MB 到 GB 级)
功耗管理	Tickless Idle (深度睡眠感知)	挂起到内存 / 空闲状态
典型应用	电机控制, 传感器节点, 安全关键系统	智能网关, HMI 界面, 路由器

表 3: 主流嵌入式平台概览

操作系统	类型	核心优势	主要应用行业
FreeRTOS	RTOS	极简内核, 生态庞大, 易上手	消费电子, 简单 IoT 节点
Zephyr	RTOS	模块化, 连接性强, 开源治理	可穿戴设备, 智能家居, 复杂 IoT
QNX	微内核	通过安全认证, 自愈能力, 虚拟化	汽车座舱, 医疗设备, 轨道交通
VxWorks	RTOS	硬实时性能, 长期可靠性验证	航空航天, 国防, 工业机器人

Yocto	构建系统	定制化 Linux 发行版生成	工业 HMI, 电信基站
-------	------	-----------------	--------------

引用的著作

1. c10-Non-Conventional OS.pdf
2. Comparative Study of Different Mobile Operating Systems | by Rautrenuka - Medium, 访问时间为 一月 9, 2026, <https://medium.com/@rautrenuka317/comparative-study-of-different-mobile-operating-systems-51017e127cbd>
3. The Android Booting process - NXP Community, 访问时间为 一月 9, 2026, <https://community.nxp.com/t5/i-MX-Processors-Knowledge-Base/The-Android-Booting-process/ta-p/1129182>
4. Architecture overview - Android Open Source Project, 访问时间为 一月 9, 2026, <https://source.android.com/docs/core/architecture>
5. What the Zygote!?. Android OS boot process explained with... | by Miso Ivanisevic | Medium, 访问时间为 一月 9, 2026, <https://medium.com/@voodoomio/what-the-zygote-76f852d887d9>
6. Study of Kernels in Different Operating Systems in Mobile Devices - Journal of Online Engineering Education, 访问时间为 一月 9, 2026, <https://onlineengineeringeducation.com/index.php/joee/article/download/82/80>
7. Solved: Compare the architecture of Android and iOS Mobile Operating Systems., 访问时间为 一月 9, 2026, <https://www.atlas.org/solution/e66d6321-b26a-4775-b747-041d0a82db42/compare-the-architecture-of-android-and-i-os-mobile-operating-systems>
8. IOS vs Android app architecture similarities | by Matan Cohen Abravanel - Medium, 访问时间为 一月 9, 2026, <https://medium.com/@matanabravanel/ios-vs-android-app-architecture-similarities-5f91cc7d4296>
9. The Risks of Android OS Fragmentation in Enterprises - Trio MDM, 访问时间为 一月 9, 2026, <https://www.trio.so/blog/android-os-fragmentation>
10. Android Fragmentation state. Did Google fixed it? - ProAndroidDev, 访问时间为 一月 9, 2026, <https://proandroiddev.com/android-fragmentation-state-did-google-fixed-it-0e7be1649e66>
11. Android Project Mainline: Everything you need to know - Hexnode, 访问时间为 一月 9, 2026, <https://www.hexnode.com/blogs/android-project-mainline-everything-you-need-to-know/>
12. How Project Mainline is Solving Android's Fragmentation Problem - Esper.io, 访问时间为 一月 9, 2026, <https://www.esper.io/blog/what-is-project-mainline>
13. Mainline | Android Open Source Project, 访问时间为 一月 9, 2026, <https://source.android.com/docs/core/ota/modular-system>
14. What's New in iOS and Android App Development for 2025 - Winklix -

- WordPress.com, 访问时间为 一月 9, 2026,
<https://winklixblog.wordpress.com/whats-new-in-ios-and-android-app-development-for-2025/>
15. Android 15 features and changes list - Android Developers, 访问时间为 一月 9, 2026, <https://developer.android.com/about/versions/15/summary>
 16. Secure Enclave - Apple Support, 访问时间为 一月 9, 2026, <https://support.apple.com/guide/security/secure-enclave-sec59b0b31ff/web>
 17. Reverse Engineering iOS 18 Inactivity Reboot - cat /dev/brain, 访问时间为 一月 9, 2026, <https://naehrdine.blogspot.com/2024/11/reverse-engineering-ios-18-inactivity.html>
 18. Samsung Knox vs. iOS Security: A Cybersecurity Engineer's Deep Dive - Medium, 访问时间为 一月 9, 2026, <https://medium.com/@samyakgoel5/samsung-knox-vs-ios-security-a-cybersecurity-engineers-deep-dive-80b5652a7a6e>
 19. Optimize for Doze and App Standby | App quality - Android Developers, 访问时间为 一月 9, 2026, <https://developer.android.com/training/monitoring-device-state/doze-standby>
 20. Platform power management with Doze | Android Open Source Project, 访问时间为 一月 9, 2026, https://source.android.com/docs/core/power/platform_mgmt
 21. Overview of memory management | App quality - Android Developers, 访问时间为 一月 9, 2026, <https://developer.android.com/topic/performance/memory-overview>
 22. Real-time operating system - Wikipedia, 访问时间为 一月 9, 2026, https://en.wikipedia.org/wiki/Real-time_operating_system
 23. Comprehensive Analysis of FreeRTOS Task Scheduling and Priority Management: From Tick to Delay - Oreate AI Blog, 访问时间为 一月 9, 2026, <https://www.oreateai.com/blog/comprehensive-analysis-of-freertos-task-scheduling-and-priority-management-from-tick-to-delay/29d677497a212a9fb8f5553e3b083715>
 24. Android OS - Processes Scheduling, 访问时间为 一月 9, 2026, <https://darajat.ly/wp-content/uploads/2024/06/L4-Android-OS-Processes-Scheduling-V2.pdf>
 25. What's difference between MMU and MPU? - GeeksforGeeks, 访问时间为 一月 9, 2026, <https://www.geeksforgeeks.org/operating-systems/whats-difference-between-mmu-and-mpu/>
 26. Automotive Memory Protection Units: Uncovering Hidden Vulnerabilities - PlaxidityX, 访问时间为 一月 9, 2026, <https://plaxidityx.com/blog/blog-post/is-your-memory-protecteduncovering-hidden-vulnerabilities-in-automotive-mpu-mechanisms/>
 27. FreeRTOS vs Linux for Embedded Systems - ByteSnap, 访问时间为 一月 9, 2026, <https://www.bytesnap.com/news-blog/freertos-vs-linux-embedded-systems/>
 28. Zephyr RTOS vs FreeRTOS: A Comprehensive Comparison for IoT and Embedded Systems, 访问时间为 一月 9, 2026,

- <https://www.ezurio.com/resources/blog/zephyr-rtos-vs-freertos-a-comprehensive-comparison-for-iot-and-embedded-systems>
29. VxWorks - Wikipedia, 访问时间为 一月 9, 2026,
<https://en.wikipedia.org/wiki/VxWorks>
 30. Zephyr Overview - 2025, 访问时间为 一月 9, 2026,
<https://www.zephyrproject.org/wp-content/uploads/2025/06/Zephyr-Overview-20250626.pdf>
 31. How Zephyr became the leading open-source RTOS for IoT - nordicsemi.com, 访问时间为 一月 9, 2026,
<https://www.nordicsemi.com/Resources/Training/Webinars/2025/How-Zephyr-became-the-leading-open-source-RTOS-for-IoT>
 32. Release notes for 5.0 (scarthgap) - the Yocto Project Documentation, 访问时间为 一月 9, 2026,
<https://docs.yoctoproject.org/migration-guides/release-notes-5.0.html>
 33. The Yocto Project, 访问时间为 一月 9, 2026, <https://www.yoctoproject.org/>
 34. Compounded Real-Time Operating Systems for Rich Real-Time Applications - IEEE Xplore, 访问时间为 一月 9, 2026,
<https://ieeexplore.ieee.org/document/10870260/>
 35. Real-Time Linux vs. RTOS for industrial networking - RT-Labs, 访问时间为 一月 9, 2026,
https://rt-labs.com/fieldbus_in_software/real-time-linux-vs-rtos-for-industrial-networking/
 36. In surgical robotics, buying time can save lives, explains BlackBerry QNX, 访问时间为 一月 9, 2026,
<https://www.therobotreport.com/in-surgical-robotics-buying-time-can-save-lives-explains-blackberry-qnx/>
 37. In Surgical Robotics, Buying Time Saves Lives - BlackBerry Blog, 访问时间为 一月 9, 2026,
<https://blogs.blackberry.com/en/2024/08/in-robotics--buying-time-saves-lives>
 38. Low power support - FreeRTOS - AWS Documentation, 访问时间为 一月 9, 2026,
<https://docs.aws.amazon.com/freertos/latest/userguide/low-power-support.html>
 39. Low Power Support - FreeRTOS™, 访问时间为 一月 9, 2026,
<https://freertos.org/low-power-tickless-rtos.html>
 40. How will the Cyber Resilience Act impact embedded developers? - EE World Online, 访问时间为 一月 9, 2026,
<https://www.eeworldonline.com/how-will-the-cyber-resilience-act-impact-embedded-developers/>
 41. EU Cyber Resilience Act - Infineon Technologies, 访问时间为 一月 9, 2026,
<https://www.infineon.com/product-information/eu-cra>
 42. QNX vs AAOS (Android Automotive OS): Compare the Platforms, 访问时间为 一月 9, 2026, <https://androidautomotive.dev/blog/qnx-vs-aaos>
 43. Android Automotive - Wikipedia, 访问时间为 一月 9, 2026,
https://en.wikipedia.org/wiki/Android_Automotive
 44. Automotive Focus: BlackBerry's QNX Driving into the Future - ENERGYDM Group, 访问时间为 一月 9, 2026,

<https://energydmgroup.com/automotive-focus-blackberrys-qnx-driving-into-the-future/>

45. Foundational Software Solutions for Medical Devices - QNX, 访问时间为 一月 9, 2026, <https://qnx.software/en/industries/medical-devices>
46. VxWorks for Medical Devices Solution Brief - Wind River Systems, 访问时间为 一月 9, 2026, <https://www.windriver.com/resource/vxworks-for-medical-devices-solution-brief>
47. Why Matter Protocol is The Future of IoT Connectivity? - Oxeltech, 访问时间为 一月 9, 2026, <https://oxeltech.de/why-matter-protocol-is-the-future-of-iot-connectivity/>
48. Simplifying Smart Homes: Learn How Matter, Thread and Wi-Fi are Revolutionizing IoT Connectivity - Qorvo, 访问时间为 一月 9, 2026, <https://www.qorvo.com/design-hub/blog/simplifying-smart-homes-learn-how-matter-thread-and-wi-fi-are-revolutionizing-iot-connectivity>
49. TinyML at the Edge: Deploying and Optimizing AI Workloads on Zephyr RTOS - Amandeep Singh, Welzin - YouTube, 访问时间为 一月 9, 2026, https://www.youtube.com/watch?v=o-kD8bG__fo
50. Machine Learning with Conexio Stratus, Zephyr RTOS and Edge Impulse, 访问时间为 一月 9, 2026, <https://zephyrproject.org/machine-learning-with-conexio-stratus-zephyrrtos-and-edge-impulse/>